EV Battery Equivalent Circuit and SOC Estimation

This example derives the Thevenin 2-RC equivalent circuit model of a lithium-ion cell from first principles, builds a Laplace-domain impedance model for EIS validation, linearizes the SOC-dependent dynamics for observer design, and generates compact functions that can be reused in downstream BMS workflows.

The equivalent circuit model is the workhorse of battery management: two coupled first-order ODEs describe polarization voltage dynamics, while SOC integrates coulombs. The Thevenin model is simple enough to derive symbolically but rich enough for production BMS. The symbolic approach delivers exact analytical forms for impedance spectra, observer Jacobians, and parameter sensitivities that numeric-only workflows cannot.

Requires: Symbolic Math Toolbox™

Copyright 2026 The MathWorks, Inc.

1. Thevenin 2-RC Equivalent Circuit Model

1.1 Declare Symbolic Parameters

All cell parameters remain symbolic so that the voltage response, impedance spectrum, and observer equations emerge as formulas. Substitute realistic NMC pouch cell parameters only for validation. The 2-RC model captures both fast charge-transfer dynamics ($\tau_1 \sim 1 - 10$ s) and slow diffusion dynamics ($\tau_2 \sim 30 - 300$ s).

```
syms R0 R1 C1 R2 C2 positive          % circuit element parameters
syms Q_nom positive                    % nominal cell capacity [A*s]
syms t real                            % time [s]
syms s                                 % Laplace variable
syms tau real                          % auxiliary time variable for impulse response
syms I_app real                        % applied current [A] (positive = discharge)
```

Parameter definitions:

- R_0 -- series (ohmic) resistance: contact resistance + electrolyte bulk [1--3 mΩ for high-power EV cells]
- R_1, C_1 -- fast RC pair: charge-transfer at electrode/electrolyte interface [$\tau_1 = R_1 C_1 \sim 2$ s]
- R_2, C_2 -- slow RC pair: solid-state diffusion in active material [$\tau_2 = R_2 C_2 \sim 60$ s]
- Q_{nom} -- nominal capacity [A·s] (e.g., 75 Ah = 270000 A·s for an EV cell)

1.2 RC Branch Dynamics via `dsolve`

Each RC branch obeys a first-order ODE: the voltage across the parallel R-C pair responds to applied current with time constant $\tau = RC$. This is the governing physics of polarization buildup and relaxation.

ODE for RC branch k: $C_k \frac{dV_k}{dt} + \frac{V_k}{R_k} = I_{app}$

```
syms V1(t) V2(t)
```

RC branch 1 ODE:

```
ode1 = C1*diff(V1, t) + V1/R1 == I_app
```

```
ode1(t) =
```

$$C_1 \frac{\partial}{\partial t} V_1(t) + \frac{V_1(t)}{R_1} = I_{\text{app}}$$

RC branch 2 ODE:

```
ode2 = C2*diff(V2, t) + V2/R2 == I_app
```

```
ode2(t) =
```

$$C_2 \frac{\partial}{\partial t} V_2(t) + \frac{V_2(t)}{R_2} = I_{\text{app}}$$

1.3 Solve RC Dynamics -- Exact Time-Domain Response

Apply `dsolve` with zero initial polarization (cell at equilibrium before current step). The solution reveals the exponential buildup with time constant $\tau_k = R_k C_k$.

```
V1_sol = dsolve(ode1, V1(0) == 0);
V2_sol = dsolve(ode2, V2(0) == 0);
```

V1(t) -- fast polarization voltage:

```
V1_sol = simplify(V1_sol)
```

```
V1_sol =
```

$$I_{\mathrm{app}} R_1 - I_{\mathrm{app}} R_1 \, \mathrm{e}^{-\frac{t}{C_1 R_1}}$$

V2(t) -- slow polarization voltage:

```
V2_sol = simplify(V2_sol)
```

```
V2_sol =
```

$$I_{\mathrm{app}} R_2 - I_{\mathrm{app}} R_2 \, \mathrm{e}^{-\frac{t}{C_2 R_2}}$$

1.4 Terminal Voltage Expression

The cell terminal voltage is:

$$V_{term}(t) = V_{OC}(SOC) - R_0 I_{app} - V_1(t) - V_2(t)$$

where $V_{OC}(SOC)$ is the open-circuit voltage. The negative signs follow the convention that $I_{app} > 0$ during discharge (terminal voltage drops below OCV).

```
syms V_OC real
V_term = V_OC - R0*I_app - V1_sol - V2_sol;
V_term = simplify(V_term)
```

```
V_term =
```

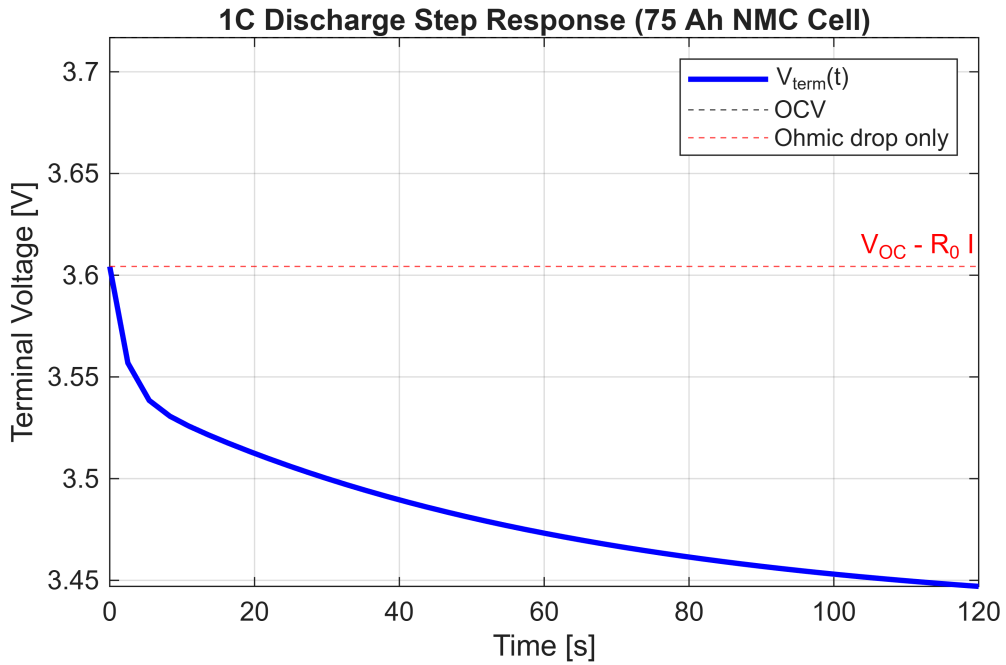
$$V_{\mathrm{OC}} - I_{\mathrm{app}} R_0 - I_{\mathrm{app}} R_1 - I_{\mathrm{app}} R_2 + I_{\mathrm{app}} R_1 \, \mathrm{e}^{-\frac{t}{C_1 R_1}} + I_{\mathrm{app}} R_2 \, \mathrm{e}^{-\frac{t}{C_2 R_2}}$$

The result shows the superposition structure: an instantaneous ohmic drop $R_0 I$ plus two exponential relaxation terms with time constants $R_1 C_1$ and $R_2 C_2$.

1.5 Validate Against Current Step Response

Substitute representative NMC pouch cell parameters (75 Ah EV cell, BOL at 25 degC) and plot the voltage response to a 1C discharge step.

```
R0_val  = 0.0015;    % 1.5 mOhm series resistance (modern high-power EV cell, BOL 25 degC)
R1_val  = 0.0008;    % 0.8 mOhm charge-transfer resistance
C1_val  = 2500;      % 2500 F (tau1 = 2.0 s)
R2_val  = 0.0015;    % 1.5 mOhm diffusion resistance
C2_val  = 40000;     % 40000 F (tau2 = 60 s)
Q_nom_val = 270000;  % 75 Ah in A*s
I_1C = 75;           % 1C rate for 75 Ah cell
z_nom = 0.5;         % 50% SOC operating point
V_OC_nom = 3.45 + 0.667*(z_nom - 0.1);
V_term_Val = subs(V_term, [R0, R1, C1, R2, C2, I_app, V_OC], ...
    [R0_val, R1_val, C1_val, R2_val, C2_val, I_1C, V_OC_nom]);
fplot(V_term_Val, [0 120], 'b-', 'LineWidth', 2)
yline(V_OC_nom, 'k--', 'V_{OC}');
yline(V_OC_nom - R0_val*I_1C, 'r--', 'V_{OC} - R_0 I');
xlabel('Time [s]')
ylabel('Terminal Voltage [V]')
title('1C Discharge Step Response (75 Ah NMC Cell)')
grid on
legend('V_{term}(t)', 'OCV', 'Ohmic drop only', 'Location', 'northeast')
```



2. Impedance Spectroscopy Model (Laplace Domain)

2.1 Cell Impedance via Laplace Transform

Electrochemical impedance spectroscopy (EIS) measures the cell's frequency response. For each RC branch, the current-to-voltage impulse response is $(1/C_k)e^{-t/(R_kC_k)}$. Taking its Laplace transform gives the impedance contribution $R_k/(1+sR_kC_k)$, and the series resistance adds a real offset.

```
z_RC1 = (1/C1) * exp(-tau/(R1*C1));
z_RC2 = (1/C2) * exp(-tau/(R2*C2));
Z_RC1 = laplace(z_RC1, tau, s);
Z_RC2 = laplace(z_RC2, tau, s);
Z_cell = simplify(R0 + Z_RC1 + Z_RC2)
```

```
Z_cell =
```

$$R_0 + \frac{R_1}{C_1 R_1 s + 1} + \frac{R_2}{C_2 R_2 s + 1}$$

2.2 Partial Fraction Decomposition -- Separate the Contributions

Apply `partfrac` to reveal how each electrochemical process contributes independently to the impedance. Each term maps to a semicircle in the Nyquist plot.

```
Z_pf = partfrac(Z_cell, s)
```

```
Z_pf =
```

$$R_0 + \frac{R_1}{C_1 R_1 s + 1} + \frac{R_2}{C_2 R_2 s + 1}$$

The decomposition shows three distinct contributions: R_0 (pure real, no frequency dependence), a pole at $s = -1/\tau_1$ (fast charge-transfer semicircle), and a pole at $s = -1/\tau_2$ (slow diffusion semicircle). Each term is independently identifiable from EIS data.

2.3 Impulse Response via Inverse Laplace

The time-domain impulse response $z(t) = \mathscr{L}^{-1}\{Z(s)\}$ is the voltage response per unit current impulse. This connects the frequency-domain impedance model back to the time-domain step response derived in Section 1.

```
z_impulse = ilaplace(Z_cell, s, tau);
z_impulse = simplify(z_impulse)
```

```
z_impulse =
```

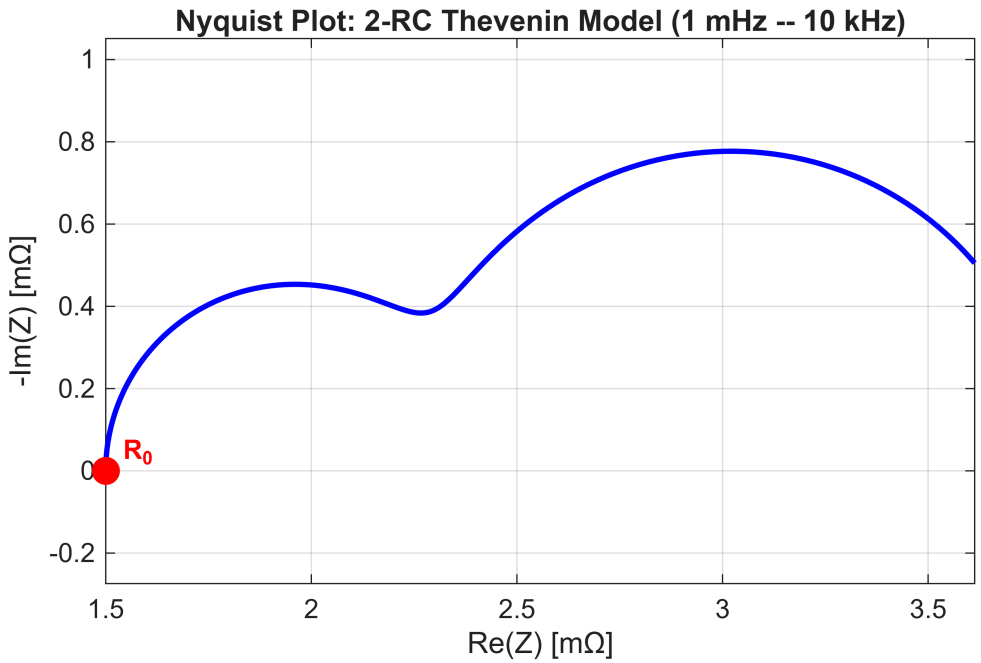
$$R_0\,\delta(\tau) + \frac{\mathrm{e}^{-\frac{\tau}{C_1 R_1}}}{C_1} + \frac{\mathrm{e}^{-\frac{\tau}{C_2 R_2}}}{C_2}$$

The Dirac delta from R_0 plus two decaying exponentials from the RC pairs -- consistent with the `dsolve` result, now derived independently via the Laplace path.

2.4 Nyquist Plot -- Validate Impedance Model Against Typical EIS Data

Substitute cell parameters and sweep frequency from 1 mHz to 10 kHz. The resulting Nyquist plot should show two overlapping semicircles -- the signature of a 2-RC Thevenin model.

```
f_sweep = logspace(-3, 4, 500);
omega_sweep = 2*pi*f_sweep;
Z_Val = subs(Z_cell, [R0, R1, C1, R2, C2], [R0_val, R1_val, C1_val, R2_val, C2_val]);
Z_Vals = double(subs(Z_Val, s, 1i*omega_sweep));
plot(real(Z_Vals)*1000, -imag(Z_Vals)*1000, 'b-', 'LineWidth', 2)
hold on
plot(R0_val*1000, 0, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r')
hold off
xlabel('Re(Z) [m\Omega]')
ylabel('-Im(Z) [m\Omega]')
title('Nyquist Plot: 2-RC Thevenin Model (1 mHz -- 10 kHz)')
grid on
axis equal
text(R0_val*1000, 0, '  R_0', 'Color', 'r', ...
    'FontWeight', 'bold', 'VerticalAlignment', 'bottom')
```



The high-frequency intercept gives R_0, the first semicircle diameter gives R_1, and the second gives R_2. Time constants determine the frequency at each semicircle's apex: $f_k = 1/(2\pi R_k C_k)$.

3. SOC-Dependent Parameters via `piecewise`

3.1 OCV-SOC Relationship

The open-circuit voltage is a strongly nonlinear function of state of charge. For NMC cells, the OCV curve has three distinct regions: a steep low-SOC region (intercalation onset), a flat mid-range plateau, and a steep high-SOC region (approaching full lithiation). Model this with a `piecewise` function that captures the essential shape.

```
syms z real                              % SOC (0 to 1)
assume(z >= 0 & z <= 1)
V_OC_sym = piecewise( ...
    z < 0.1,  3.0 + 4.5*z, ...
    z < 0.9,  3.45 + 0.667*(z - 0.1), ...
    z >= 0.9, 3.45 + 0.667*0.8 + 2.2*(z - 0.9))
```

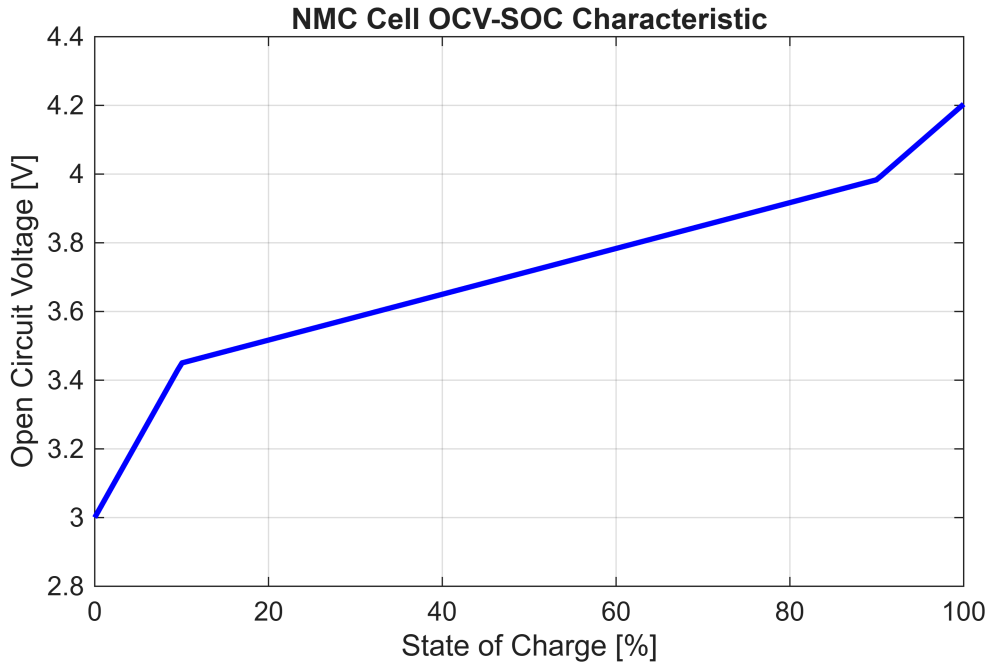
```
V_OC_sym =
```

$$\begin{cases} \frac{9z}{2} + 3 & \text{if } z < \frac{1}{10} \\ \frac{667z}{1000} + \frac{33833}{10000} & \text{if } z < \frac{9}{10} \\ \frac{11z}{5} + \frac{5009}{2500} & \text{if } \frac{9}{10} \le z \end{cases}$$

3.2 Plot OCV Curve

```
z_plot = linspace(0, 1, 200);
V_OC_plot = double(subs(V_OC_sym, z, z_plot));
plot(z_plot*100, V_OC_plot, 'b-', 'LineWidth', 2)
xlabel('State of Charge [%]')
ylabel('Open Circuit Voltage [V]')
title('NMC Cell OCV-SOC Characteristic')
grid on
```

```
ylim([2.8 4.4])
```



The piecewise OCV model captures the key feature: the mid-range plateau (10--90% SOC) where dV_{OC}/dz is small makes SOC estimation from voltage alone nearly impossible, thus motivating model-based observers.

3.3 SOC-Dependent Internal Resistance

Internal resistance increases at SOC extremes due to increased charge-transfer overpotential (low SOC) and lithium plating risk management (high SOC). Model $R_0(z)$ as piecewise with elevated values at the extremes.

```
syms R0_mid R0_factor_low R0_factor_high positive
R0_SOC = piecewise( ...
    z < 0.1,  R0_mid * R0_factor_low, ...
    z < 0.9,  R0_mid, ...
    z >= 0.9, R0_mid * R0_factor_high)
```

```
R0_SOC =
```

$$\begin{cases} R_{0,\text{mid}}\, R_{0,\text{factor,low}} & \text{if } z < \frac{1}{10} \\ R_{0,\text{mid}} & \text{if } z < \frac{9}{10} \\ R_{0,\text{mid}}\, R_{0,\text{factor,high}} & \text{if } \frac{9}{10} \le z \end{cases}$$

```
R0_mid_val = R0_val;
R0_factor_low_val = 1.5;
R0_factor_high_val = 1.3;
```

Typical values: $R_{0,mid} = 1.5$ mΩ, low-SOC factor $= 1.5$, high-SOC factor $= 1.3$. This SOC-dependent resistance directly affects the voltage prediction accuracy of the observer. The same symbolic `piecewise` expressions are reused later when generating the voltage prediction function.

4. Linearization for Real-Time SOC Estimation

4.1 State-Space Model

The battery state vector is $x = [z, V_1, V_2]^T$ with dynamics:

$$\dot{z} = -I_{app}/Q_{nom}$$

$$\dot{V}_1 = -V_1/(R_1C_1) + I_{app}/C_1$$

$$\dot{V}_2 = -V_2/(R_2C_2) + I_{app}/C_2$$

$$V_{term} = V_{OC}(z) - R_0I_{app} - V_1 - V_2$$

This is the standard form for extended Kalman filter (EKF) based SOC estimation.

```
syms V1_state V2_state real            % polarization voltages as states
syms I_sym real                        % current (input)
f_state = [-I_sym/Q_nom;
           -V1_state/(R1*C1) + I_sym/C1;
           -V2_state/(R2*C2) + I_sym/C2]
```

```
f_state =
```

$$\begin{bmatrix} -\frac{I_{\mathrm{sym}}}{Q_{\mathrm{nom}}} \\ \frac{I_{\mathrm{sym}}}{C_1} - \frac{V_{1,\mathrm{state}}}{C_1 R_1} \\ \frac{I_{\mathrm{sym}}}{C_2} - \frac{V_{2,\mathrm{state}}}{C_2 R_2} \end{bmatrix}$$

4.2 Taylor Expansion of OCV -- Linearization About Operating Point

The output equation is nonlinear through $V_{OC}(z)$. For EKF design, we need the output Jacobian $\partial h/\partial z = dV_{OC}/dz$. Apply `taylor` to the OCV function in the mid-SOC region (where EV cells spend most of their operating life) to obtain a local linear approximation.

```
syms z0 real                          % linearization point
assume(z0 > 0.1 & z0 < 0.9)
```

In the mid-range (0.1 < z < 0.9), the OCV model gives:

```
V_OC_mid = 3.45 + 0.667*(z - 0.1);
```

Taylor expansion of V_OC about z0 (mid-range):

```
V_OC_taylor = taylor(V_OC_mid, z, z0, 'Order', 2)
```

```
V_OC_taylor =
```

$$\frac{667\,z}{1000} + \frac{33833}{10000}$$

dV_OC/dz in mid-range:

```
dVOC_dz_mid = diff(V_OC_mid, z)
```

```
dVOC_dz_mid =
```

$$\frac{667}{1000}$$

This small value (compared to ~ 4.5 V at low SOC) is precisely why voltage-only SOC estimation fails in the plateau region - the signal-to-noise ratio for SOC observability collapses.

4.3 Taylor Expansion at SOC Extremes

At low SOC, the steep OCV provides much better observability. Expand the low-SOC branch to show the contrast.

```
V_OC_low = 3.0 + 4.5*z;
```

dV_OC/dz at low SOC:

```
dVOC_dz_low = diff(V_OC_low, z)
```

```
dVOC_dz_low =
```

$$\frac{9}{2}$$

```
fprintf('Observability ratio (low SOC / mid SOC): %.1fx\n', 4.5/0.667)
```

```
Observability ratio (low SOC / mid SOC): 6.7x
```

The low-SOC region has 6.7x better voltage-based SOC observability -- this is why production BMS systems switch estimation strategies near SOC extremes.

4.4 System Jacobians for EKF

The extended Kalman filter requires the state Jacobian $A = \partial f/\partial x$ and output Jacobian $C = \partial h/\partial x$. Compute these symbolically using `jacobian`.

```
x_state = [z; V1_state; V2_state];
```

State Jacobian A = df/dx:

```
A_sym = jacobian(f_state, x_state)
```

```
A_sym =
```

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & -\frac{1}{C_1 R_1} & 0 \\ 0 & 0 & -\frac{1}{C_2 R_2} \end{bmatrix}$$

Output equation: $h(x,u) = V_{OC}(z) - V_1 - V_2 - R_0 I$

```
h_output = V_OC_mid - V1_state - V2_state - R0*I_sym;
```

Output Jacobian C = dh/dx:

```
C_sym = jacobian(h_output, x_state)
```

```
C_sym =
```

$$\begin{bmatrix} \frac{667}{1000} & -1 & -1 \end{bmatrix}$$

Key insight: A is diagonal (states are decoupled in the dynamics) and C contains dV_{OC}/dz in the first element. This is the observability link between voltage measurement and SOC. When $dV_{OC}/dz \to 0$ (mid-SOC plateau), the system becomes nearly unobservable from voltage alone.

5. Analytical Parameter Sensitivities

5.1 Voltage Sensitivity to R1

How does terminal voltage error propagate from uncertainty in the charge-transfer resistance? This exact sensitivity formula guides the calibration engineer on which parameters most affect voltage prediction accuracy.

```
V_term_sym = V_OC_mid - R0*I_sym - V1_state - V2_state;
dV_dR1 = diff(V1_sol, R1);
```

dV1/dR1 -- voltage sensitivity to charge-transfer resistance:

```
dV_dR1 = simplify(dV_dR1)
```

```
dV_dR1 =
```

$$I_{\text{app}} - I_{\text{app}}\, \mathrm{e}^{-\frac{t}{C_1 R_1}} - \frac{I_{\text{app}}\, t\, \mathrm{e}^{-\frac{t}{C_1 R_1}}}{C_1 R_1}$$

5.2 SOC Estimation Error Sensitivity to Capacity Fade

As the cell ages, Q_{nom} decreases. The SOC integration $\dot{z} = -I/Q$ means capacity uncertainty directly biases the SOC estimate. Compute the sensitivity of SOC to capacity error.

```
syms Q_true Q_est positive
syms t_end positive
z_true = -I_app*t_end / Q_true;
z_est  = -I_app*t_end / Q_est;
z_error = z_est - z_true;
```

SOC error sensitivity to capacity estimate:

```
dz_dQ = simplify(diff(z_error, Q_est))
```

```
dz_dQ =
```

$$\frac{I_{\text{app}}\, t_{\text{end}}}{Q_{\text{est}}^{\,2}}$$

At $t = 3600$ s with 1C current: a 5% capacity error ($Q_{est}/Q_{true} = 0.95$) accumulates SOC error:

```
z_err_Val = double(subs(z_error, [I_app, t_end, Q_true, Q_est], ...
    [I_1C, 3600, Q_nom_val, 0.95*Q_nom_val]));
fprintf('SOC drift after 1 hour at 1C with 5%% capacity error: %.1f%%n', z_err_Val*100)
```

```
SOC drift after 1 hour at 1C with 5% capacity error: -5.3%n
```

This drift rate is why production BMS must continuously adapt Q_{nom} -- the symbolic sensitivity formula tells you exactly how fast the estimate degrades.

5.3 Impedance Sensitivity to Temperature (via R0)

Internal resistance follows Arrhenius behavior with temperature. The sensitivity of terminal voltage to R_0 variations gives the temperature-induced voltage prediction error.

dV_term/dR0 -- terminal voltage sensitivity to series resistance:

```
dVterm_dR0 = diff(V_term_sym, R0)
```

dVterm_dR0 = $-I_{\mathrm{sym}}$

```
fprintf('At 75 A (1C): 1 mOhm R0 error causes %.1f mV voltage error\n', abs(double(subs(dVterm_dR0, I_sym, I_1C))))
```

```
At 75 A (1C): 1 mOhm R0 error causes 75.0 mV voltage error
```

The sensitivity is simply $-I_{app}$: at higher currents, resistance uncertainty matters more. This is why fast-charging conditions demand better thermal models.

6. Linearized Observer for Real-Time SOC Estimation

6.1 Discrete-Time State-Space (ZOH)

For embedded BMS running at sample rate T_s, discretize the continuous system using a zero-order hold. The diagonal structure of A makes the matrix exponential trivial.

```
syms Ts positive
A_Val = subs(A_sym, [R1, C1, R2, C2, Q_nom], ...
    [R1_val, C1_val, R2_val, C2_val, Q_nom_val]);
```

Discrete-time A matrix (exact ZOH):

```
Ad = simplify(expm(A_Val * Ts))
```

Ad =

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & e^{-\frac{Ts}{2}} & 0 \\ 0 & 0 & e^{-\frac{Ts}{60}} \end{bmatrix}$$

The diagonal structure means each state decays independently: SOC integrates exactly, and each RC voltage decays as e^{-T_s/τ_k}.

6.2 Input Matrix Discretization

Since the A matrix is singular (SOC row is a pure integrator with zero eigenvalue), we compute $B_d = \int_0^{T_s} e^{A\tau}\, d\tau\, B$ directly. The diagonal structure makes each integral elementary.

```
B_cont = [-1/Q_nom; 1/C1; 1/C2];
B_Val = subs(B_cont, [Q_nom, C1, C2], [Q_nom_val, C1_val, C2_val]);
syms tau_int positive
integrand = simplify(expm(A_Val * tau_int) * B_Val);
```

Discrete-time B matrix:

```
Bd = simplify(int(integrand, tau_int, 0, Ts))
```

```
Bd =
```

$$\begin{bmatrix} -\frac{Ts}{270000} \\ \frac{1}{1250} - \frac{e^{-\frac{Ts}{2}}}{1250} \\ \frac{3}{2000} - \frac{3\,e^{-\frac{Ts}{60}}}{2000} \end{bmatrix}$$

6.3 Observer Gain Structure

The Luenberger observer (or steady-state Kalman filter) corrects the state prediction using the voltage innovation $e_k = V_{meas} - \hat{V}_{term}$:

$$\hat{x}_{k+1} = A_d\hat{x}_k + B_d I_k + L(V_{meas,k} - C\hat{x}_k - DI_k)$$

The symbolic work provides the exact A_d, B_d, and output Jacobian C. The observer gain L is normally tuned or calibrated downstream, so this example keeps it as a small numeric design choice rather than expanding into a full Kalman filter lesson.

```
C_obs = double(C_sym)
```

```
C_obs = 1×3
    0.6670   -1.0000   -1.0000
```

Observability check: rank of $[C; CA_d; CA_d^2]$

```
Ad_Val = double(subs(Ad, Ts, 0.1));
obsv_matrix = [C_obs; C_obs*Ad_Val; C_obs*Ad_Val^2];
fprintf('Observability rank (should be 3): %d\n', rank(obsv_matrix))
```

```
Observability rank (should be 3): 3
```

The local linearized system is observable from voltage measurements alone, though observability degrades in the mid-SOC plateau as predicted by the dV_{OC}/dz analysis.

6.4 Simulate SOC Estimation with Observer

Run a constant-current discharge with the Luenberger observer correcting a 10% SOC initialization error. Use $T_s = 0.1$ s (10 Hz, typical BMS rate).

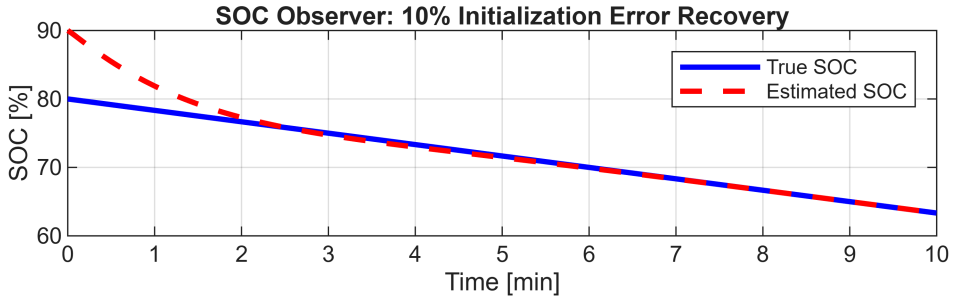
```
Ts_val = 0.1;
Ad_sim = double(subs(Ad, Ts, Ts_val));
Bd_sim = double(subs(Bd, Ts, Ts_val));
L_sim = [0.0015; 0.015; 0.0004];  % Damped teaching gain; tune downstream for a target BMS
N_steps = 6000;
t_sim = (0:N_steps-1) * Ts_val;
x_true = [0.8; 0; 0];
x_hat  = [0.9; 0; 0];
I_profile = I_1C * ones(1, N_steps);
soc_true_hist = zeros(1, N_steps);
soc_est_hist  = zeros(1, N_steps);
V_true_hist   = zeros(1, N_steps);
```

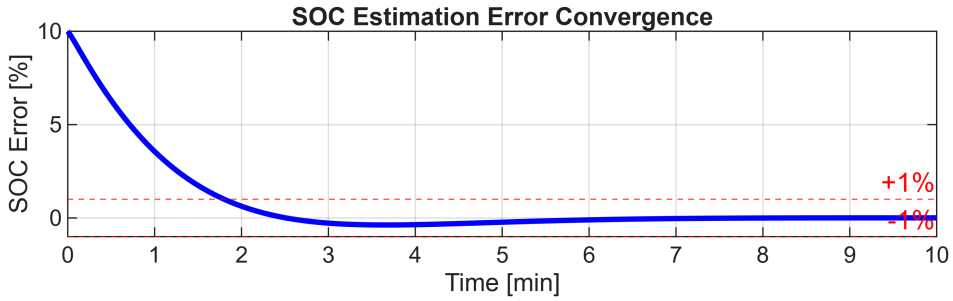
```
for k = 1:N_steps
    soc_true_hist(k) = x_true(1);
    soc_est_hist(k)  = x_hat(1);
    soc_k = x_true(1);
    if soc_k < 0.1
        V_oc_k = 3.0 + 4.5*soc_k;
    elseif soc_k < 0.9
        V_oc_k = 3.45 + 0.667*(soc_k - 0.1);
    else
        V_oc_k = 3.45 + 0.667*0.8 + 2.2*(soc_k - 0.9);
    end
    V_meas = V_oc_k - R0_val*I_profile(k) - x_true(2) - x_true(3);
    V_true_hist(k) = V_meas;
    soc_hat_k = x_hat(1);
    if soc_hat_k < 0.1
        V_oc_hat = 3.0 + 4.5*soc_hat_k;
    elseif soc_hat_k < 0.9
        V_oc_hat = 3.45 + 0.667*(soc_hat_k - 0.1);
    else
        V_oc_hat = 3.45 + 0.667*0.8 + 2.2*(soc_hat_k - 0.9);
    end
    V_pred = V_oc_hat - R0_val*I_profile(k) - x_hat(2) - x_hat(3);
    innovation = V_meas - V_pred;
    x_true = Ad_sim * x_true + Bd_sim * I_profile(k);
    x_hat  = Ad_sim * x_hat  + Bd_sim * I_profile(k) + L_sim * innovation;
end
```

6.5 Observer Convergence Results

```
soc_error = soc_est_hist - soc_true_hist;
settling_tol = 0.01;
outside_band = find(abs(soc_error) > settling_tol);
if isempty(outside_band)
    settling_time = 0;
elseif outside_band(end) < numel(t_sim)
    settling_time = t_sim(outside_band(end) + 1);
else
```

```
    settling_time = NaN;
end
soc_undershoot = max(0, -min(soc_error)) * 100;
tiledlayout(2,1)
nexttile
plot(t_sim/60, soc_true_hist*100, 'b-', t_sim/60, soc_est_hist*100, 'r--', 'LineWidth', 2)
xlabel('Time [min]')
ylabel('SOC [%]')
title('SOC Observer: 10% Initialization Error Recovery')
legend('True SOC', 'Estimated SOC', 'Location', 'northeast')
grid on
nexttile
plot(t_sim/60, soc_error*100, 'b-', 'LineWidth', 2)
xlabel('Time [min]')
ylabel('SOC Error [%]')
title('SOC Estimation Error Convergence')
grid on
yline(1, 'r--', '+1%');
yline(-1, 'r--', '-1%');
```






```
fprintf(['Initial SOC error: %.1f%%n' ...
    'Final SOC error: %.3f%%n' ...
    'Settling time within +/-1%% band: %.1f s\n' ...
    'Peak undershoot after correction: %.2f%%n'], ...
    (soc_est_hist(1) - soc_true_hist(1))*100, ...
    (soc_est_hist(end) - soc_true_hist(end))*100, ...
    settling_time, ...
    soc_undershoot)
```

```
Initial SOC error: 10.0%nFinal SOC error: 0.004%nSettling time within +/-1% band: 108.1 s
Peak undershoot after correction: 0.38%n
```

7. Code Generation for BMS Algorithm Handoff

7.1 Generate Voltage Prediction Function

Convert the symbolic terminal voltage model to a MATLAB® function file. This generated function reuses the `piecewise` OCV and SOC-dependent resistance expressions from Section 3, so the downstream observer receives the same model structure that was derived symbolically.

```
V_term_codegen = V_OC_sym - R0_SOC*I_sym - V1_state - V2_state;
matlabFunction(V_term_codegen, 'File', 'predictTerminalVoltage', ...
    'Vars', {z, V1_state, V2_state, I_sym, R0_mid, R0_factor_low, R0_factor_high}, ...
    'Outputs', {'V_pred'}, ...
    'Optimize', true);
fprintf('\n--- Generated predictTerminalVoltage.m ---\n')
```

```
--- Generated predictTerminalVoltage.m ---
```

```
type predictTerminalVoltage.m
```

```
function V_pred = predictTerminalVoltage(z,V1_state,V2_state,I_sym,R0_mid,R0_factor_low,R0_factor_high)
%predictTerminalVoltage
%    V_pred = predictTerminalVoltage(Z,V1_state,V2_state,I_sym,R0_mid,R0_factor_low,R0_factor_high)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    11-Aug-2026 11:09:33

t2 = -V1_state;
t3 = -V2_state;
if ~all(cellfun(@isscalar,{z}))
    error(message('symbolic:sym:matlabFunction:ConditionsMustBeScalar'));
end
if (z < 1.0./1.0e+1)
    V_pred = t2+t3+z.*(9.0./2.0)-I_sym.*R0_mid.*R0_factor_low+3.0;
elseif (z < 9.0./1.0e+1)
    V_pred = t2+t3+z.*(6.67e+2./1.0e+3)-I_sym.*R0_mid+3.3833;
elseif (9.0./1.0e+1 <= z)
    V_pred = t2+t3+z.*(1.1e+1./5.0)-I_sym.*R0_mid.*R0_factor_high+2.0036;
else
    V_pred = NaN;
end
end
```

7.2 Generate Output Jacobian Function

An EKF needs the measurement Jacobian at each update. Generate the slope schedule from the symbolic OCV branches instead of hand-coding it downstream. This keeps the implementation compact while avoiding an undefined derivative exactly at the resistance breakpoints.

```
dVOC_dz_high = diff(3.45 + 0.667*0.8 + 2.2*(z - 0.9), z);
```

```
dVOC_dz_codegen = piecewise( ...
    z < 0.1, dVOC_dz_low, ...
    z < 0.9, dVOC_dz_mid, ...
    z >= 0.9, dVOC_dz_high);
C_output_codegen = [dVOC_dz_codegen, -1, -1];
matlabFunction(C_output_codegen, 'File', 'batteryOutputJacobian', ...
    'Vars', {z, I_sym, R0_mid, R0_factor_low, R0_factor_high}, ...
    'Outputs', {'C_out'}, ...
    'Optimize', true);
fprintf('\n--- Generated batteryOutputJacobian.m ---\n')
```

```
--- Generated batteryOutputJacobian.m ---
```

```
type batteryOutputJacobian.m
```

```
function C_out = batteryOutputJacobian(z,I_sym,R0_mid,R0_factor_low,R0_factor_high)
%batteryOutputJacobian
%    C_out = batteryOutputJacobian(Z,I_sym,R0_mid,R0_factor_low,R0_factor_high)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    11-Aug-2026 11:09:33

if ~all(cellfun(@isscalar,{z}))
    error(message('symbolic:sym:matlabFunction:ConditionsMustBeScalar'));
end
if (z < 1.0./1.0e+1)
    t0 = 9.0./2.0;
elseif (z < 9.0./1.0e+1)
    t0 = 6.67e+2./1.0e+3;
elseif (9.0./1.0e+1 <= z)
    t0 = 1.1e+1./5.0;
else
    t0 = NaN;
end
C_out = [t0,-1.0,-1.0];
end
```

7.3 Generate State Update Function

The discrete-time state transition for the EKF prediction step comes from the exact ZOH matrices derived in Section 6. Numeric cell parameters are baked in here to keep the generated function small; parameterized variants follow the same `matlabFunction` pattern.

```
f_discrete_sym = sym(Ad_sim) * [z; V1_state; V2_state] + sym(Bd_sim) * I_sym;
```

```
matlabFunction(f_discrete_sym, 'File', 'batteryStateUpdate', ...
    'Vars', {z, V1_state, V2_state, I_sym}, ...
    'Outputs', {'x_next'}, ...
    'Optimize', true);
fprintf('\n--- Generated batteryStateUpdate.m ---\n')
```

```
--- Generated batteryStateUpdate.m ---
```

```
type batteryStateUpdate.m
```

```
function x_next = batteryStateUpdate(z,V1_state,V2_state,I_sym)
%batteryStateUpdate
%    X_NEXT = batteryStateUpdate(Z,V1_state,V2_state,I_sym)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    11-Aug-2026 11:09:33

x_next = [I_sym.*(-3.703703703703704e-7)+z;I_sym.*3.901646039942879e-5+V1_state.*9.51229424500714e-1;I_sym.*2.497917823591982e-6+V2_state.*9.98334721450
end
```

7.4 Optional C Export for Third-Party Workflows

`ccode` is not the Embedded Coder handoff. It is useful when you need source-level arithmetic for a third-party firmware stack, a calibration tool, or an independent implementation review. The Embedded Coder path starts from the generated MATLAB functions in a Simulink MATLAB Function block, as shown in Section 8.2.

First export C arithmetic for the impedance model. Since the impedance is a rational function of s, substituting $s = j\omega$ and rationalizing separates real and imaginary parts algebraically.

```
syms omega real positive
tau1 = R1*C1;
tau2 = R2*C2;
Z_re = R0 + R1/(1 + omega^2*tau1^2) + R2/(1 + omega^2*tau2^2);
Z_im = -omega*R1*tau1/(1 + omega^2*tau1^2) - omega*R2*tau2/(1 + omega^2*tau2^2);
```

Z_re(omega):

```
Z_re
```

```
Z_re =
```

$$R_0 + \frac{R_1}{C_1^2 R_1^2 \omega^2 + 1} + \frac{R_2}{C_2^2 R_2^2 \omega^2 + 1}$$

Z_im(omega):

```
Z_im
```

```
Z_im =
```

$$-\frac{C_1 R_1^2 \omega}{C_1^2 R_1^2 \omega^2 + 1} - \frac{C_2 R_2^2 \omega}{C_2^2 R_2^2 \omega^2 + 1}$$

```
fprintf('\n--- C code: Real part of impedance Z_re(omega) ---\n')
```

```
--- C code: Real part of impedance Z_re(omega) ---
```

```
ccode(Z_re)
```

```
ans =
'  t0 = R0+R1/((C1*C1)*(R1*R1)*(omega*omega)+1.0)+R2/((C2*C2)*(R2*R2)*(omega*omega)+1.0);'
```

```
fprintf('\n--- C code: Imaginary part of impedance Z_im(omega) ---\n')
```

```
--- C code: Imaginary part of impedance Z_im(omega) ---
```

```
ccode(Z_im)
```

```
ans =
'  t0 = -(C1*(R1*R1)*omega)/((C1*C1)*(R1*R1)*(omega*omega)+1.0)-(C2*(R2*R2)*omega)/((C2*C2)*(R2*R2)*(omega*omega)+1.0);'
```

7.5 Optional C Export for State Update

Export the state prediction arithmetic as a small C expression block for third-party or hand-integrated workflows. For MathWorks code generation, use `batteryStateUpdate` in a Simulink MATLAB Function block and let Embedded Coder generate the target code from that block.

```
fprintf('\n--- C code: Battery state prediction (discretized) ---\n')
```

```
--- C code: Battery state prediction (discretized) ---
```

```
ccode(f_discrete_sym)
```

```
ans =
    '  f_discrete_sym[0][0] = I_sym*(-3.703703703703704E-7)+z;
       f_discrete_sym[1][0] = I_sym*3.901646039942879E-5+V1_state*9.51229424500714E-1;
       f_discrete_sym[2][0] = I_sym*2.497917823591982E-6+V2_state*9.983347214509387E-1;'
```

8. Downstream Workflow Connections

8.1 EIS Validation -- Fit Symbolic Impedance Model to Measurement Data

The symbolic impedance $Z(s)$ from Section 2 provides the reference model for fitting EIS measurement data. Here we simulate a realistic scenario: generate synthetic EIS data (true parameters + measurement noise), then recover the parameters by fitting the symbolic model.

```
Z_model_fun = matlabFunction(Z_cell, 'Vars', {R0, R1, C1, R2, C2, s});
```

Synthesize "measured" EIS data: true cell parameters with 2% Gaussian noise (representing typical potentiostat measurement uncertainty at these impedance magnitudes).

```
rng(42)
f_meas = logspace(-2, 3, 40);
omega_meas = 2*pi*f_meas;
Z_true = Z_model_fun(R0_val, R1_val, C1_val, R2_val, C2_val, 1i*omega_meas);
noise_level = 0.02;
Z_meas = Z_true .* (1 + noise_level*(randn(size(Z_true)) + 1i*randn(size(Z_true))));
```

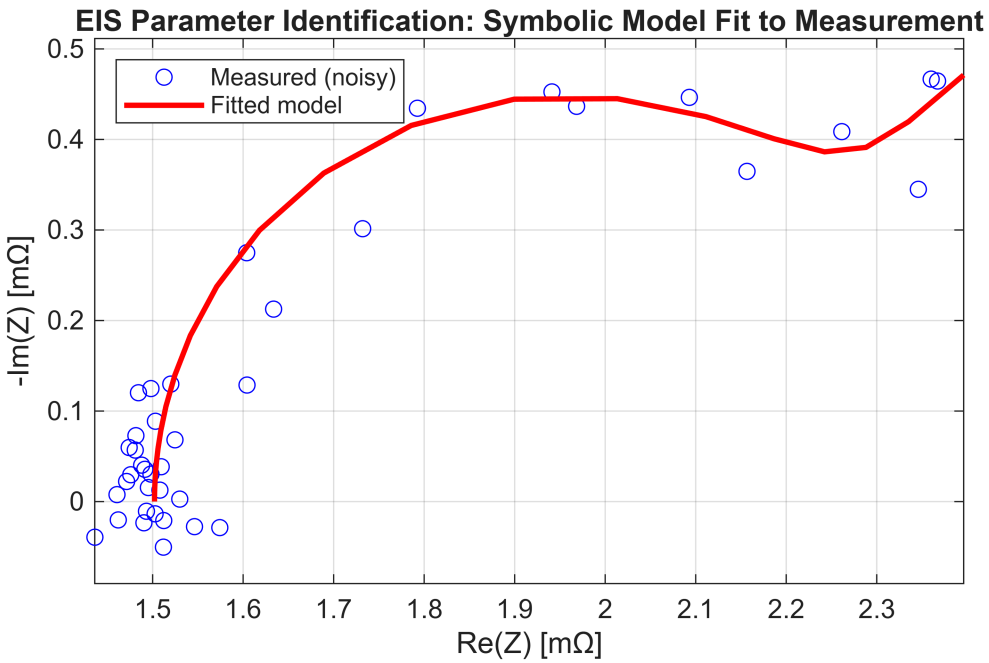
Fit parameters using `fminsearch` (base MATLAB). The cost function minimizes the sum of squared impedance residuals across all frequencies.

```
cost = @(p) sum(abs(Z_model_fun(p(1),p(2),p(3),p(4),p(5), 1i*omega_meas) - Z_meas).^2);
p0 = [0.002, 0.001, 2000, 0.002, 30000];
p_fit = fminsearch(cost, p0, optimset('TolFun', 1e-20, 'TolX', 1e-10, 'MaxFunEvals', 5000));
fprintf(['Parameter recovery from noisy EIS data:\n' ...
    '  R0: true = %.4f mOhm, fit = %.4f mOhm\n' ...
    '  R1: true = %.4f mOhm, fit = %.4f mOhm\n' ...
    '  C1: true = %.0f F,    fit = %.0f F\n' ...
    '  R2: true = %.4f mOhm, fit = %.4f mOhm\n' ...
    '  C2: true = %.0f F,  fit = %.0f F\n'], ...
    R0_val*1e3, p_fit(1)*1e3, ...
    R1_val*1e3, p_fit(2)*1e3, ...
```

```
    C1_val, p_fit(3), ...
    R2_val*1e3, p_fit(4)*1e3, ...
    C2_val, p_fit(5))
```

```
Parameter recovery from noisy EIS data:
  R0: true = 1.5000 mOhm, fit = 1.5021 mOhm
  R1: true = 0.8000 mOhm, fit = 0.7844 mOhm
  C1: true = 2500 F,    fit = 2550 F
  R2: true = 1.5000 mOhm, fit = 1.2714 mOhm
  C2: true = 40000 F,  fit = 38466 F
```

```
Z_fit = Z_model_fun(p_fit(1), p_fit(2), p_fit(3), p_fit(4), p_fit(5), 1i*omega_meas);
clf
plot(real(Z_meas)*1e3, -imag(Z_meas)*1e3, 'bo', 'MarkerSize', 6)
hold on
plot(real(Z_fit)*1e3, -imag(Z_fit)*1e3, 'r-', 'LineWidth', 2)
hold off
xlabel('Re(Z) [m\Omega]')
ylabel('-Im(Z) [m\Omega]')
title('EIS Parameter Identification: Symbolic Model Fit to Measurement')
legend('Measured (noisy)', 'Fitted model', 'Location', 'northwest')
grid on
axis equal
```



8.2 Simulink Deployment -- Real-Time SOC Observer

The generated functions (`predictTerminalVoltage`, `batteryOutputJacobian`, `batteryStateUpdate`) form the compact computational core for a Simulink MATLAB Function block:

```
% Inside a Simulink MATLAB Function block (10 Hz execution):
function [SOC_est, V_pred, C_out] = socObserverStep(V_meas, I_meas, x_hat_prev, L_gain)
  R0_mid = 0.0015; lowFactor = 1.5; highFactor = 1.3;
  x_pred = batteryStateUpdate(x_hat_prev(1), x_hat_prev(2), x_hat_prev(3), I_meas);
  V_pred = predictTerminalVoltage(x_pred(1), x_pred(2), x_pred(3), I_meas, R0_mid, lowFactor, highFactor);
  C_out = batteryOutputJacobian(x_pred(1), I_meas, R0_mid, lowFactor, highFactor);
  innovation = V_meas - V_pred;
  x_hat = x_pred + L_gain * innovation;
  SOC_est = x_hat(1);
end
```

In a fixed-gain observer, `L_gain` is calibrated outside this block. In an EKF, `C_out` feeds the covariance and gain update.

8.3 Simscape Battery Validation

Simscape Battery is the high-fidelity plant in a validation model, and the generated MATLAB functions are wrapped in a Simulink MATLAB Function block that implements the reduced-order observer:

- **Plant subsystem:** Simscape Battery cell or pack model with electrical and thermal dynamics
- **Observer subsystem:** Simulink MATLAB Function block calling `batteryStateUpdate`, `predictTerminalVoltage`, and `batteryOutputJacobian`
- **Validation signal flow:** feed plant current and terminal voltage into the observer; compare predicted voltage and estimated SOC against plant outputs across drive cycles

The symbolic derivation keeps the observer equations traceable to the circuit assumptions; Simscape validation then tests whether the reduced-order model and calibrated parameters are accurate over the intended operating envelope.

8.4 Deployment Path Summary

```
Symbolic Thevenin 2-RC Model (this example)
    |
    +--> dsolve: exact time-domain response for step/pulse test fitting
    |
    +--> laplace/partfrac: impedance model for EIS parameter identification
    |
    +--> taylor/jacobian: linearized observer for EKF-based SOC estimation
    |
    +--> matlabFunction: generated MATLAB functions
    |         |
    |         +--> Simulink MATLAB Function block observer
    |                   |
    |                   +--> Embedded Coder: target C for BMS ECU (ARM Cortex-M4)
    |                   |
    |                   +--> Simscape Battery validation: observer runs beside plant model
    |
    +--> ccode: optional C export for third-party firmware or calibration tools
```